



**JOMO KENYATTA UNIVERSITY OF AGRICULTURE AND
TECHNOLOGY.**

**COMPUTER VISION MODULE FOR WAZIGOV: DEEP
LEARNING-BASED TABLE STRUCTURE RECOGNITION IN
GOVERNMENT FINANCIAL DOCUMENTS**

Student Name: Joseph Kariuki Wairimu

Registration Number: SCT212-0147/2022

Unit Code: BCT 2406

Course Name: Project - BSc CT (Year 4)

Date of Submission: 3/3/2026

Supervisor(s): Dr. Evanson Karanja

ABSTRACT

Government budget documents contain critical tabular data embedded in complex structures featuring hierarchical headers, merged cells, and multi-page continuity. This project develops a Computer Vision module using deep learning to extract precise table structures from Kenyan government PDF reports. The system employs a DETR-based Table Transformer architecture for detection and structure recognition, achieving pixel-perfect cell localization. The module processes PDF documents through a multi-stage pipeline:

1. Document preprocessing and image conversion.
2. Table detection using object detection transformers.
3. Structure recognition with row-column segmentation.
4. Cell grid generation with merged cell identification.
5. Validation, including arithmetic checks and hierarchical header detection.

Evaluation on 100 manually annotated government documents targets >90% F1 score for table detection and >85% GriTS score for structure recognition. This level of accuracy will enable automated analysis of previously inaccessible budget data. The system handles scanned documents, rotated tables, and complex multi-level hierarchies that traditional PDF parsing methods struggle to parse. The module outputs structured JSON with precise cell coordinates and base64-encoded cell images, forming the foundation for downstream NLP processing in the broader WaziGov transparency system.

Keywords: Computer Vision, Deep Learning, Table Detection, Structure Recognition, Document Layout Analysis, Object Detection, Transformers, DETR, Table Transformer, Financial Documents, International Public Sector Accounting Standards (IPSAS).

ABBREVIATIONS

CLAHE: Contrast Limited Adaptive Histogram Equalization

CNN: Convolutional Neural Network

CV: Computer Vision

DETR: Detection Transformer

DPI: Dots Per Inch

FFN: Feed-Forward Network

GriTS: Grid Table Similarity

GPU: Graphics Processing Unit

HTAN: Hierarchical Table-Aware Network

ICDAR: International Conference on Document Analysis and Recognition

IoU: Intersection over Union

IPSAS: International Public Sector Accounting Standards

JSON: JavaScript Object Notation

MLLM: Multimodal Large Language Model

NLP: Natural Language Processing

NMS: Non-Maximum Suppression

OCR: Optical Character Recognition

PDF: Portable Document Format

REST: Representational State Transfer

RPN: Region Proposal Network

TATR: Table Transformer

TSR: Table Structure Recognition

TABLE OF CONTENTS

1. INTRODUCTION
 - 1.1 Background and Context
 - 1.2 Problem Statement
 - 1.3 Proposed Solution
 - 1.4 Objectives
2. STATE OF THE ART
 - 2.1 Traditional Table Detection Methods
 - 2.2 Deep Learning Approaches
 - 2.3 Transformer-Based Architectures
 - 2.4 Financial Document Processing
 - 2.5 Research Gaps
3. APPROACH/METHODOLOGY
 - 3.1 Description of the Computer Vision Pipeline
 - 3.1.1 Overview
 - 3.1.2 System Design and Architectural Decisions
 - 3.1.3 Stage 1: Document Preprocessing
 - 3.1.4 Stage 2: Table Detection Model
 - 3.1.5 Stage 3: Structure Recognition
 - 3.1.6 Stage 4: Domain-Specific Post-Processing
 - 3.1.7 Model Fine-Tuning for Domain Adaptation
 - 3.1.8 Stage 5: JSON Output Schema
 - 3.2 Technology Stack Summary
 - 3.3 Data
 - 3.4 Evaluation Strategy and Metrics
 - 3.5 Expected Outcomes
 - 3.6 Ethical Considerations
4. CONCLUSION
5. REFERENCES
6. APPENDICES
 - Appendix 1: Project Plan and Timeline (Gantt Chart)
 - Appendix 2: Budget (Itemized)

1. INTRODUCTION

1.1 Background and Context

The Broader WaziGov System

This project is part of WaziGov, an integrated AI pipeline designed to extract, process, and visualize financial data from Kenyan government budget documents. The system comprises three primary modules:

1. **Computer Vision Module (this project)** - Extracts table structures from PDF documents
2. **Natural Language Processing Module** - Extracts text entities from cell images
3. **Geospatial Intelligence Module** - Maps projects to administrative boundaries

This proposal focuses exclusively on the Computer Vision module, which serves as the critical first stage in the pipeline. The module's primary responsibility is converting unstructured PDF documents into structured, machine-readable table representations with precise geometric information.

Document Characteristics

Kenyan government budget documents follow International Public Sector Accounting Standards (IPSAS) and exhibit specific structural patterns:

- **Hierarchical Headers:** 3-5 levels of nested headers (e.g., "Development Budget" -> "Infrastructure" -> "Roads" -> "Maintenance")
- **Merged Cells:** Column and row spans for category headers
- **Multi-Page Tables:** Tables spanning 10-50 pages with continuation patterns
- **Mixed Quality:** Both digital PDFs and scanned documents (varying DPI: 150-600)
- **Complex Layouts:** Tables interspersed with text blocks, page numbers, watermarks
- **Numerical Precision:** Financial amounts requiring exact extraction

The Computing Challenge

Traditional PDF parsing libraries (PyPDF2, pdfplumber, tabula-py) operate on text coordinates and spatial heuristics. Schreiber et al. (2017) demonstrate that these methods fail on government documents because:

1. **No Visual Understanding:** Cannot "see" table boundaries as images; only process text positions, rendering them blind to visual structure cues (Schreiber et al., 2017).
2. **Merged Cell Blindness:** Lack mechanisms to detect cells spanning multiple rows/columns, resulting in incorrect fragmentation (Chi et al., 2019).
3. **Scanned Document Failure:** Return empty outputs when no embedded text layer exists, common in archived documents (Schreiber et al., 2017).

4. Continuation Detection: Cannot identify multi-page table relationships, treating each page independently (Qiao et al., 2021).
5. Structure Loss: Flatten hierarchical headers into linear text streams, losing essential parent-child relationships (Schreiber et al., 2017).

These limitations result in accuracy rates of only 30-40% on complex hierarchical tables (Schreiber et al., 2017), far below production requirements.

1.2 Problem Statement

Core Technical Problem: Document Layout Analysis for Complex Tables

The primary computing challenge is automated table structure recognition in visually complex, hierarchically organized financial documents. Specifically:

Problem 1: Table Detection in Cluttered Documents

Given a document page image $I \in \mathbb{R}^{(H \times W \times 3)}$, identify all table regions $T = \{t_1, t_2, \dots, t_n\}$ where each t_i is defined by bounding box coordinates (x_1, y_1, x_2, y_2) .

Challenges: - Tables lack clear visual boundaries (no borders in some cases) - Large whitespace areas that are not table separators - Text blocks resembling single-column tables - Page headers/footers/watermarks overlapping with tables

Problem 2: Structure Recognition with Merged Cells

For each detected table t_i , decompose into a grid structure $G(R, C, M)$ where:

- $R = \{r_1, r_2, \dots, r_m\}$ are row boundaries
- $C = \{c_1, c_2, \dots, c_n\}$ are column boundaries
- $M = \{(i,j,h,w)\}$ are merged cells spanning h rows and w columns

Challenges: - Rows with varying heights (header rows taller than data rows) - Columns with inconsistent widths across rows - Merged cells creating ambiguous grid intersections - Hierarchical headers requiring parent-child relationship inference

Problem 3: Multi-Page Table Continuity

Detect when table t_i on page p continues as table t_j on page $p+1$, requiring: - Column width matching across pages - Header row identification and exclusion - Seamless row index continuation.

Research Question

Can a DETR-based transformer architecture, pre-trained on scientific document tables, be effectively adapted to Kenyan government financial documents with complex hierarchies through domain-specific fine-tuning and post-processing, achieving production-level accuracy of greater than 90% F1 for table detection and greater than 85% GriTS for structure recognition?

1.3 Proposed Solution

The Computer Vision module implements a five-stage pipeline that combines pre-trained transformer models with domain-specific post-processing logic. Stage 1 is document preprocessing, in which raw PDFs are converted to 300 DPI page images, subjected to skew correction, and contrast-normalised for scanned pages. Stage 2 is table detection, in which the Table Transformer detection model identifies table bounding boxes across each page. Stage 3 is structure recognition, in which cropped table images are processed by the Table Transformer structure model to produce row, column, and spanning cell predictions. Stage 4 is grid construction and post-processing, encompassing merged cell resolution, hierarchical header detection, and multi-page continuity stitching. Stage 5 is validation and serialisation, producing a structured JSON document consumed by the downstream NLP module.

The core architecture is built on the Detection Transformer (DETR; Carion et al., 2020), which reformulates object detection as a direct set prediction problem. DETR uses a CNN backbone followed by a transformer encoder-decoder to predict object locations without hand-crafted components such as anchor boxes or non-maximum suppression, making it well suited to the variable-scale, often borderless tables characteristic of government documents.

Microsoft’s Table Transformer (Smock et al., 2022) adapts DETR for table understanding through two complementary models. The detection model accepts a full document page image ($H \times W \times 3$) as input and outputs table bounding boxes with class labels identifying standard and rotated tables; it was pre-trained on PubTables-1M (947,000 annotated tables). The structure recognition model accepts a cropped table image as input and produces cell-level predictions across five classes: table row, table column, table spanning cell, table projected row header, and table column header. Both models are fine-tuned on an annotated dataset of Kenyan government document pages using a two-stage domain adaptation strategy described in Chapter 3.

The module’s output is a machine-readable JSON document containing, for each extracted table: bounding box coordinates and page number, a cell grid with row and column indices, merged cell span annotations (row-span and col-span), hierarchy depth levels for header cells, base64-encoded cell images for downstream NLP processing, and confidence scores and arithmetic validation flags.

1.4 Objectives

1.4.1 Main Objective

To develop a deep learning-based Computer Vision module that accurately detects and extracts table structures from Kenyan government financial documents, achieving >90% F1 score on

table detection and >85% GriTS score on structure recognition while handling hierarchical headers, merged cells, and multi-page continuity.

1.4.2 Specific Objectives

Research and Analysis:

- i. To conduct a comprehensive literature review of Document Layout Analysis techniques, comparing traditional methods (heuristic, rule-based) with modern deep learning approaches (CNNs, Transformers) on financial document benchmarks.
- ii. To analyse the structural characteristics of IPSAS-compliant Kenyan government tables, documenting patterns of hierarchical headers, merged cells, and numerical formatting conventions.

Dataset Creation:

- iii. To create an annotated benchmark dataset of 100 Kenyan government document pages with ground-truth labels for table boundaries, row/column structures, and cell-level annotations. This dataset will contain approximately 150-200 table bounding boxes and an estimated 15,000-20,000 cell-level structure labels, following the annotation standard established by PubTables-1M.

Model Development:

- iv. To implement and configure Table Transformer models for table detection and structure recognition, including hyperparameter tuning and threshold optimization for government documents.
- v. To develop custom post-processing algorithms for merged cell identification, hierarchical header detection, and multi-page table continuity recognition.
- vi. To design and implement validation logic, including arithmetic consistency checks (row sums, column totals) and hierarchical relationship verification.

System Integration:

- vii. To build a modular pipeline architecture with clear input/output interfaces, enabling integration with downstream NLP and geospatial modules via RESTful APIs.
- viii. To implement efficient batch processing capabilities supporting parallel processing of large document collections with GPU acceleration.

Evaluation and Optimization:

- ix. To evaluate model performance using standard metrics (Precision, Recall, F1, IoU) and table-specific metrics (GriTS), comparing results against baseline methods (pdfplumber, Camelot). The arithmetic validator will be evaluated on its precision and recall in detecting subtotal/total inconsistencies, with a target of >95% for both metrics.

- x. To conduct error analysis identifying failure modes and proposing mitigation strategies for edge cases (low-quality scans, unusual layouts, damaged documents).

Documentation:

- xi. To produce comprehensive technical documentation including API specifications, configuration guides, and reproducibility instructions enabling independent verification of results.

2. STATE OF THE ART

2.1 Traditional Table Detection Methods

Traditional table detection relies on rule-based heuristics rather than learned representations. Schreiber et al. (2017) comprehensively evaluate classical methods based on geometric heuristics and edge detection, demonstrating that they fail on borderless tables and require manual threshold tuning for each document type.

Modern rule-based tools (pdfplumber, Camelot, tabula-py) extract text coordinates and apply spatial heuristics to infer table structure. However, these approaches exhibit critical limitations: no visual understanding of document layout, incorrect parsing of merged cells, and failure on scanned PDFs without text layers. Chi et al. (2019) highlight that traditional methods struggle with hierarchical headers, inconsistent column widths, and borderless tables where structure is implied visually rather than explicitly marked.

Schreiber et al. (2017) report that traditional approaches achieve only 30-40% accuracy on complex hierarchical tables, demonstrating the fundamental inadequacy of hand-crafted features for capturing real-world document complexity and motivating the development of deep learning methods. Recent surveys by Maurya & Prajapati (2025) and Giovannini & Marinai (2025) confirm that while rule-based methods persist in production systems, they are systematically outperformed by learning-based approaches on challenging benchmarks.

2.2 Deep Learning Approaches

CNN-Based Object Detection

DeepDeSRT (Schreiber et al., 2017) introduced the first end-to-end deep learning approach using VGG-16 backbone with Region Proposal Network (RPN) for table detection and deformable convolutions for row/column detection, achieving 91.4% F1 on ICDAR 2013. However, it requires separate models for detection vs. structure with slow inference (5-10s per page).

TableNet (Paliwal et al., 2020) unified architecture using encoder-decoder with VGG-19, implementing semantic segmentation for pixel-wise classification. It achieves 88.3% F1 on Marmot dataset but struggles with precise cell boundaries and merged cells are not explicitly modelled.

End-to-End Recognition Frameworks

More recent work has explored fully end-to-end architectures that bypass intermediate representations. E2eTRNet (2025, cited in Taverne et al., 2025) proposes a framework that jointly optimizes detection and structure recognition, specifically designed to avoid error propagation from OCR misalignment; a relevant concern for scanned government documents. While promising, such approaches require large-scale annotated datasets that are not available for the Kenyan government domain.

2.3 Transformer-Based Architectures

DETR: Detection Transformer (Carion et al., 2020)

Paradigm shift in object detection:

- Eliminates hand-crafted components (anchors, NMS)
- Direct set prediction via bipartite matching
- Transformer encoder-decoder processes image features
- Learnable object queries attend to image regions

Architecture:

Input Image -> CNN Backbone (ResNet-50) -> Flatten + Position Encoding -> Transformer Encoder (6 layers) -> Transformer Decoder (6 layers) -> FFN (class + bbox) -> Bipartite Matching Loss

Advantages for Tables:

- Set prediction naturally handles variable table counts
- Self-attention captures long-range dependencies (crucial for large tables)
- No post-processing required (no NMS)

Table Transformer (Smock et al., 2022)

Direct application of DETR to table understanding with two-stage approach:

1. Detection Model:

- Pre-trained on PubTables-1M (947,642 tables)
- Classes: {"table", "table rotated"}
- Achieves 96.6% Precision, 95.4% Recall

2. Structure Recognition Model:

- Classes: {"table row", "table column", "table spanning cell", "table projected row header", "table column header"}
- Achieves 94.2% GriTS score

Contemporary Extensions and Validations

Recent work has both validated and extended the Table Transformer foundation. Taverne et al. (2025) in **RAPTOR: Refined Approach for Product Table Object Recognition** build directly on DETR/TATR architecture, addressing persistent challenges including "incorrect area detection and overlapping columns" through genetic algorithm-based post-processing optimization. Their work demonstrates that detection-based approaches remain highly competitive, particularly when augmented with domain-specific refinement.

Zhang et al. (2025) propose **HTAN: Hierarchical Table-Aware Network**, which introduces multi-level attention mechanisms specifically designed for tables with multi-level headers; directly relevant to IPSAS-compliant government documents. They note that "over 60% of key

metrics in financial reports are embedded in tables with multi-level headers," validating the importance of the hierarchical header detection component in this proposal.

Liu et al. (2025) present **MonkeyOCR v1.5**, introducing two techniques particularly relevant to this project: **Image-Decoupled Table Parsing** for handling embedded visual elements, and **Type-Guided Table Merging** for tables crossing page boundaries. The latter provides independent validation of the multi-page continuity stitching approach proposed in Section 3.6.3, confirming that cross-page table merging remains an active research challenge.

GriTS Metric (Grid Table Similarity):

Novel metric for table structure evaluation:

$$\text{GriTS} = (\text{Precision} + \text{Recall}) / 2$$

$$\text{Precision} = \text{Correct cell adjacencies} / \text{Predicted adjacencies}$$

$$\text{Recall} = \text{Correct cell adjacencies} / \text{Ground truth adjacencies}$$

Advantages over CNNs:

- Single model handles varying table sizes
- Attention mechanism captures hierarchical relationships
- Pre-training on 947K tables provides strong initialization
- Inference: 0.5-1.5s per page on GPU

LayoutLMv3 (Huang et al., 2022)

Multimodal pre-training for document AI with joint modelling of text, layout (bounding boxes), and image. Achieves SOTA on DocVQA (93.0% F1). Relevant for header/data cell classification after structure extraction, but requires text extraction as input (not end-to-end from raw images)

2.4 Multimodal Large Language Models for Table Understanding

The emergence of Multimodal Large Language Models (MLLMs) represents a parallel track in table understanding research. Nunes et al. (2025) provide a critical benchmark in "**Benchmarking Table Extraction: Multimodal LLMs vs Traditional OCR**," directly comparing TATR+OCR against models including GPT-4o, Phi-3 Vision, and Granite Vision. Their key finding is instructive: "deep learning computer vision techniques still have a slight edge when extracting table structural layout, but in terms of text cell content, MLLMs are far better."

This finding validates the architectural decision in the WaziGov pipeline: using a computer vision module (Table Transformer) for structure extraction, followed by a separate NLP module (LayoutLMv3) for content understanding. The hybrid approach leverages the respective strengths of each paradigm.

For the specific domain of government financial documents, MLLMs present additional challenges: they require significantly more training data, exhibit higher inference latency, and their proprietary nature (for models like GPT-4o) conflicts with the transparency goals of the

WaziGov system. The open-source, fine-tunable Table Transformer approach remains the practical choice.

2.5 Financial Document Processing

Saha et al. (2019) surveyed table extraction in financial documents, identifying challenges: - multi-column layouts (balance sheets with 5-10 columns) - Numeric precision requirements (no OCR errors tolerated) - Cross-referencing between tables - Regulatory format compliance (GAAP, IFRS, IPSAS)

Gap: Limited research exists on IPSAS-compliant documents. No published work on automated extraction of IPSAS budget tables with hierarchical headers.

2.6 Post-Processing and Optimization Techniques

Recent literature increasingly recognizes that post-processing is not a concession but an essential component of production-ready table extraction systems. Taverne et al. (2025) demonstrate that genetic algorithm-based optimization of detection outputs can reduce overlapping column errors by 15-20% on challenging industrial datasets. Liu et al. (2025) show that visual consistency-based reinforcement learning during post-processing improves cross-page table merging accuracy.

These findings support the post-processing architecture proposed in Section 3.6, where custom algorithms (Merged Cell Resolver, Hierarchical Header Detector, Multi-Page Continuity Stitcher) operate on the raw model outputs. Rather than indicating a weakness in the base model, such post-processing represents domain-specific adaptation that transforms generic table predictions into production-ready structures for government documents.

2.7 Evaluation Frameworks and Metrics

GriTS and Its Limitations

Smock et al. (2022) introduced GriTS (Grid Table Similarity) as a metric specifically designed for table structure evaluation, measuring correctness of cell adjacency relationships. It has become the de facto standard in the field.

However, recent work by Liu (2025) at HKST introduces a critical perspective. In **"FROM STRUCTURAL SIMILARITY TO INFORMATION FIDELITY: A NEW PARADIGM FOR EVALUATING TABLE RECOGNITION,"** Liu argues that structural similarity metrics (including GriTS and TEDS) are "imperfect proxies" that create a "Fidelity Gap"; models may score high on structural metrics while producing "catastrophic, information-destroying errors." The proposed alternative, TFEM (Table Fidelity Evaluation Metric), uses optimal transport to directly measure information preservation.

Implications for This Project

For the WaziGov use case, where correct cell adjacency is the minimum requirement for downstream NLP processing, GriTS-Con remains appropriate as the primary metric. However,

acknowledging this debate strengthens the evaluation framework. As Liu (2025) notes, different use cases require different fidelity criteria; budget transparency demands both structural correctness and numerical precision, which will be addressed through the arithmetic validation layer (Section 3.6.4).

2.8 Research Gaps

Gap 1: Domain Adaptation to Government Documents

- Pre-trained models trained on scientific papers (PubTables-1M)
- Government tables have deeper hierarchies (3-5 levels vs. 1-2)
- IPSAS-specific formatting conventions
- African naming conventions and terminology

Gap 2: Evaluation on Real-World African Documents

- Standard benchmarks (ICDAR, PubTables-1M) lack government financial documents
- No publicly available dataset of annotated African government tables
- Recent surveys (Maurya & Prajapati, 2025; Giovannini & Marinai, 2025) confirm this geographic and domain-specific gap
- **Contribution:** Create first benchmark dataset of Kenyan budget documents

Gap 3: Multi-Page Table Continuity

- Existing work treats each page independently
- MonkeyOCR's Type-Guided Table Merging (Liu et al., 2025) confirms this as an active research problem
- No published algorithms specifically for detecting table continuation in government financial documents
- **Novel Approach:** Column-width similarity + header detection heuristics

Gap 4: Validation Logic

- Arithmetic validation (sum checks) not addressed in literature
- Hierarchical consistency checking unexplored
- TFEM critique (Liu, 2025) highlights need for information fidelity metrics; this project's arithmetic validation directly addresses information preservation

Gap 5: End-to-End System Integration

- Most research focuses on isolated table detection/recognition

- No published work on integrating CV extraction with NLP entity extraction for government transparency
- The hybrid CV + NLP approach validated by Nunes et al. (2025) benchmarks remain unexplored in African governance contexts

Project Contributions:

This project directly addresses these gaps by:

1. Contributing the first annotated benchmark dataset for Kenyan government financial tables
2. Developing and releasing a domain-adapted Table Transformer model fine-tuned on this data
3. Proposing novel, open-source algorithms for multi-page table stitching and IPSAS-specific arithmetic validation
4. Integrating the complete solution into an end-to-end transparency pipeline, demonstrating real-world impact on government accountability
5. Providing empirical evidence on the effectiveness of detection-based TSR for African government documents, addressing the geographic gap identified in recent surveys

3. APPROACH/METHODOLOGY

3.1 Description of the Computer Vision Pipeline

3.1.1 Overview

The Computer Vision (CV) module is the foundational first stage of the WaziGov pipeline. Its sole responsibility is to accept raw government PDF documents as input and produce a structured, machine-readable JSON representation of every table contained within those documents; with precise cell-level geometry, merged-cell annotations, hierarchical header labels, and base64-encoded cell images suitable for downstream NLP processing.

The pipeline is designed around a central design principle: it must handle the full spectrum of document quality found in real Kenyan government archives, from high-resolution digital PDFs to 150-DPI scanned documents with skew, noise, and watermarks. This requirement drives every architectural decision described in this chapter.

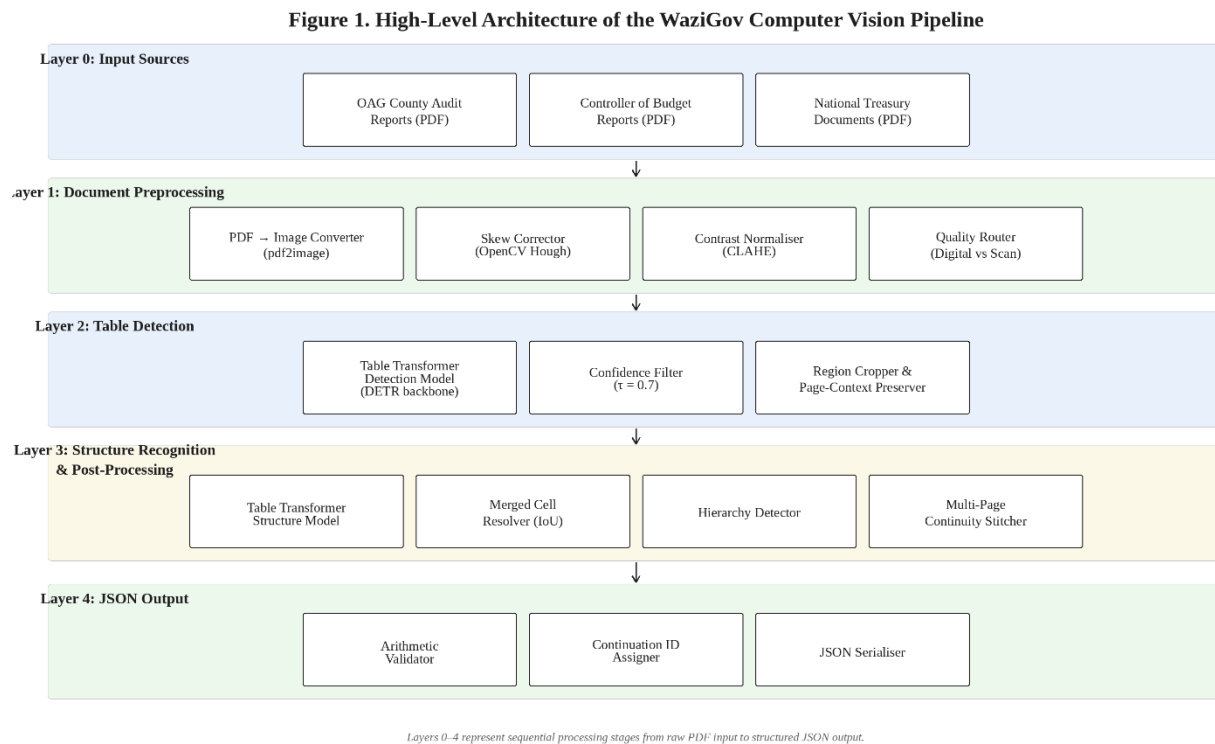


Figure 1: High-Level Architecture of the WaziGov Computer Vision Pipeline (5 Processing Layers)

As shown in Figure 1, the pipeline is organised into five sequential layers. Layer 0 ingests raw PDF reports from three primary sources. Layer 1 preprocesses each page into a normalised image tensor. Layer 2 applies the Table Transformer detection model to localise all table regions. Layer 3 performs structure recognition and domain-specific post-processing. Layer 4 validates and serialises the results to a canonical JSON schema consumed by the NLP module.

3.1.2 System Design and Architectural Decisions

Model Selection Rationale

The choice of the DETR-based Table Transformer (Smock et al., 2022) as the core detection and structure recognition model is motivated by four key properties that align with the WaziGov requirements.

- **Set-based prediction:** DETR's bipartite matching loss treats table detection as a set prediction problem, eliminating the need for hand-crafted anchor boxes and non-maximum suppression. This is particularly important for government documents where tables can appear at any scale and in any quadrant of the page.
- **Long-range self-attention:** The transformer encoder's self-attention mechanism allows the model to reason about the entire page context when predicting a single table boundary; critical for correctly excluding page headers, watermarks, and footnote blocks that share visual similarity with table rows.
- **Unified two-stage architecture:** The Table Transformer uses separate detection and structure recognition models sharing the same DETR architecture, allowing independent fine-tuning of each stage on domain-specific data without architectural changes.
- **Pre-training at scale:** Pre-training on PubTables-1M (947,642 annotated tables) provides a strong initialisation point for domain adaptation, far superior to training from scratch on a 150-page Kenyan government dataset alone.

High-Level Architecture

The high-level architecture is a linear processing pipeline with a single decision branch in Layer 1 (the Digital/Scan router) and a merge point in Layer 3 (the multi-page continuity stitcher). All other stages are sequential. This architecture prioritises simplicity and debuggability; each stage has clearly defined input and output contracts, enabling independent unit testing and ablation studies.

Low-Level Data Flow

Figure 2 presents the low-level data flow, tracing a single document page from raw pixels through each computational stage to the final JSON cell grid. Each column in the diagram represents one processing stage; horizontal arrows represent data handoffs between stages; vertical arrows represent sequential operations within a stage.

Figure 2. Low-Level Data Flow — Per-Stage Processing from PDF Input to Structured JSON Output

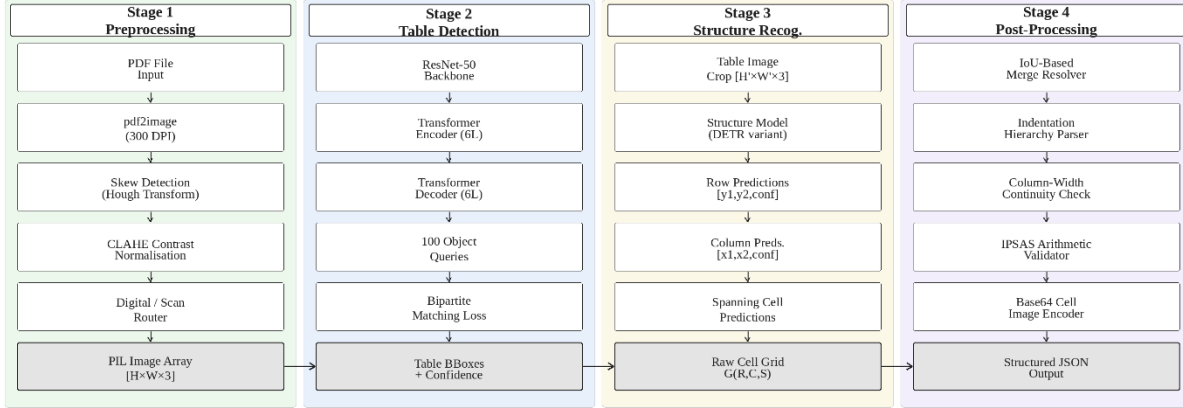


Figure 2: Low-Level Data Flow ; - Per-Stage Processing from PIL Image Array to JSON Cell Grid

The key data objects passed between stages are: a PIL Image array $[H \times W \times 3]$ from Stage 1 to Stage 2; a set of table bounding boxes with confidence scores from Stage 2 to Stage 3; a raw cell grid $G(R, C, S)$ from Stage 3 to Stage 4; and the validated, serialised JSON document object as the final output.

3.1.3 Stage 1: Document Preprocessing

PDF to Image Conversion

All input PDFs are converted to per-page PIL Image objects at 300 DPI using the pdf2image library, which wraps Poppler's pdftoppm utility. The 300 DPI resolution is chosen as it provides sufficient pixel density for the Table Transformer's ResNet-50 backbone (which internally downsamples by a factor of 32) while keeping memory consumption within the 16GB RAM constraint. At 300 DPI, an A4 page produces an image of approximately $2,480 \times 3,508$ pixels, which is subsequently resized to a maximum of 1,333 pixels on the long edge before model inference (following the standard DETR preprocessing protocol).

Skew Correction

Scanned documents frequently exhibit rotational skew introduced by misalignment during physical scanning. Uncorrected skew degrades table detection accuracy because the model's bounding box predictions assume axis-aligned tables. The skew correction algorithm uses OpenCV's Hough Line Transform on a Canny edge-detected version of the page image to estimate the dominant line angle. Correction is applied only when the detected angle exceeds ± 0.5 degrees, as sub-threshold correction on straight digital PDFs introduces interpolation artefacts that reduce downstream OCR quality.

Contrast Normalisation

Low-quality scanned documents often exhibit uneven illumination (dark borders from the scanner bed, faded central text). Contrast Limited Adaptive Histogram Equalisation (CLAHE) is applied to the luminance channel of scanned pages to normalise local contrast without amplifying noise in uniform regions. CLAHE is not applied to digital PDFs where pixel values are already uniform, as it would introduce false texture on clean backgrounds.

Digital vs. Scanned Router

The preprocessing stage includes a document quality classifier that routes pages to appropriate preprocessing paths. A page is classified as digital if the ratio of machine-readable text characters (extracted by PyMuPDF) to total page area exceeds a threshold of 0.08 characters per pixel². Pages below this threshold are routed to the scanned path, which additionally applies bilateral denoising before model inference.

Implementation Note — Preprocessing

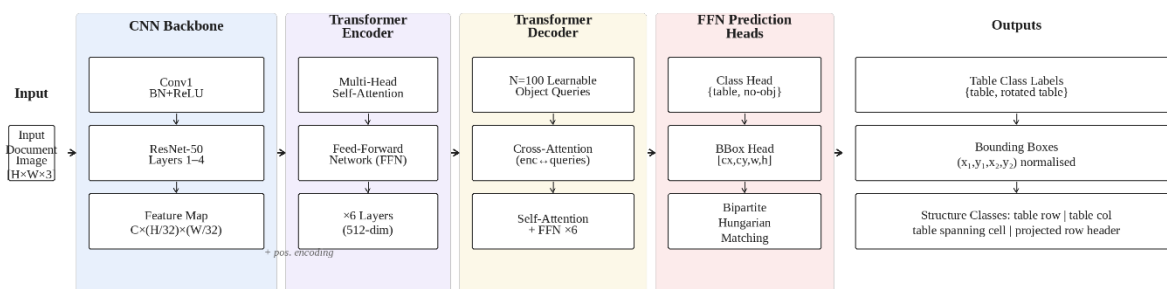
Preprocessing is implemented as a stateless Python class (DocumentPreprocessor) with a `process(pdf_path: str) -> List[PIL.Image]` interface. Each page is processed independently, enabling parallel processing across pages using Python's `multiprocessing.Pool`. GPU acceleration is not used in this stage; all operations run on CPU in under 50ms per page.

3.1.4 Stage 2: Table Detection Model

DETR Architecture Deep Dive

Figure 3 shows the complete DETR model architecture as adapted by the Table Transformer. The model processes a full-page document image through four sequential components: a CNN backbone for feature extraction, a transformer encoder for contextual feature refinement, a transformer decoder for query-based object localisation, and dual FFN prediction heads for class and bounding box output.

Figure 3. DETR-Based Table Transformer Model Architecture



Architecture follows Carion et al. (2020); adapted by Smock et al. (2022), N=100 object queries; bipartite matching eliminates NMS.

Figure 3: Complete DETR-Based Table Transformer Architecture ;:- Detection and Structure Models Share This Design

CNN Backbone

The ResNet-50 backbone extracts a feature map of shape $C \times (H/32) \times (W/32)$, where $C=2048$ is the number of channels and $H/32$, $W/32$ are the spatially downsampled height and width. A 1×1 convolutional projection reduces C to $d=256$, the transformer model dimension. Sinusoidal 2D positional encodings are added to the flattened feature sequence before feeding into the transformer encoder, preserving spatial relationships that would otherwise be lost during the flatten operation.

Transformer Encoder

The encoder consists of 6 identical layers, each comprising a multi-head self-attention sublayer (8 heads, $d_{\text{model}}=256$) followed by a feed-forward network ($d_{\text{ff}}=2048$) with residual connections and layer normalisation. The self-attention mechanism enables the model to correlate distant regions of the document page, allowing it to understand that a header row at the top of the page belongs to the same table as a data row 2,000 pixels lower.

Transformer Decoder and Object Queries

The decoder maintains $N=100$ learnable object query vectors, each representing a potential table instance. These queries attend to the encoder output via cross-attention, learning to localise specific regions of the page. The decoder also applies self-attention among the queries themselves, enabling the model to reason about the relative positions of multiple tables on the same page and avoid duplicate predictions. For typical government document pages, $N=100$ is more than sufficient as pages rarely contain more than 3-4 distinct tables.

Prediction Heads and Bipartite Matching

Two FFN heads produce per-query outputs: a class logit (table / rotated table / no-object) and a normalised bounding box (cx , cy , w , h). During training, predicted sets are matched to ground-truth sets using the Hungarian algorithm, which finds the optimal bijective assignment minimising a combined classification and localisation cost. This matching-based loss eliminates the need for NMS and enables end-to-end differentiable training. During inference, all predictions with class confidence above $\tau=0.7$ are retained as detected tables.

3.1.5 Stage 3: Structure Recognition

Structure Recognition Model

The structure recognition model shares the DETR architecture but operates on a cropped table image rather than the full page, and predicts a richer set of output classes: table row, table column, table spanning cell, table projected row header, and table column header. The model's object queries learn to specialise; some queries consistently predict row boundaries, others column boundaries, and others spanning cells; an emergent behaviour arising from the bipartite matching training objective.

For each detected table region, the cropped image is padded to a square aspect ratio and resized to 448×448 pixels before inference. This fixed-size input allows efficient batching of multiple table crops in a single forward pass, reducing total inference time by a factor of approximately $3\times$ compared to processing each crop independently.

3.5.2 Grid Construction from Structure Predictions

The structure model outputs are continuous bounding boxes rather than a discrete grid. Converting these predictions into a canonical $R \times C$ cell grid requires the following steps:

1. Sort row predictions by their vertical centre coordinate (y_centre).
2. Sort column predictions by their horizontal centre coordinate (x_centre).
3. For each (row_i , col_j) intersection, determine the occupying cell by checking which spanning cell prediction (if any) covers the intersection point, falling back to a default data cell.
4. Resolve row and column boundary conflicts using a 1D non-maximum suppression with IoU threshold 0.3 on the respective axis.
5. Assign each cell its row index r , column index c , row-span h , and column-span w , producing the final grid matrix $G(R, C)$.

3.1.6 Stage 4: Domain-Specific Post-Processing

This stage contains the novel algorithmic contributions of the project. Three parallel algorithms; the Merged Cell Resolver, the Hierarchical Header Detector, and the Multi-Page Continuity Stitcher; operate on the raw cell grid and produce an enriched, validated cell grid ready for serialisation. Figure 4 presents the complete logic flow for all three algorithms.

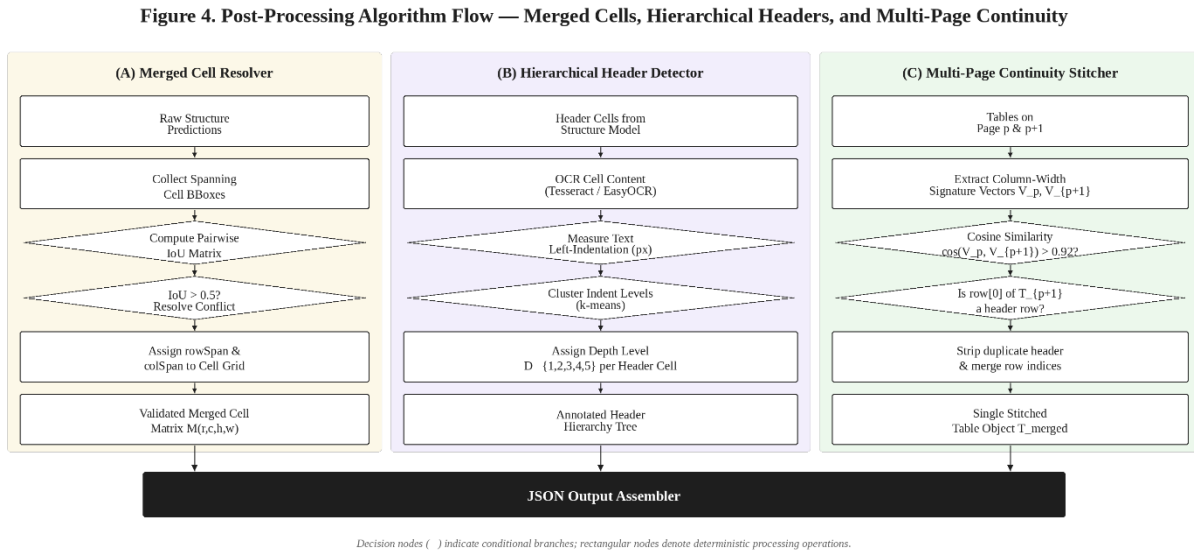


Figure 4: Post-Processing Algorithm Flow ; - Three Parallel Algorithms Enriching the Raw Cell Grid

Merged Cell Resolver (Lane A)

The Table Transformer structure model predicts spanning cells as separate bounding box objects with the class label 'table spanning cell'. However, in practice the predictions for large merged cells often overlap with the constituent row/column predictions, and multiple spanning cell predictions occasionally conflict with each other. The Merged Cell Resolver resolves these conflicts as follows.

For each pair of spanning cell predictions, the Intersection over Union (IoU) of their bounding boxes is computed. If IoU exceeds 0.50, the lower-confidence prediction is suppressed. The surviving spanning cell predictions are then projected onto the grid, and each grid cell within a spanning cell's boundary is tagged with the corresponding rowSpan and colSpan values. The resulting merged cell matrix $M(r, c, h, w)$ encodes the complete spanning structure of the table.

Hierarchical Header Detector (Lane B)

IPSAS-compliant government budget tables use 3-5 levels of nested headers to encode the budget classification hierarchy (e.g., Programme -> Sub-Programme -> Activity -> Sub-Activity). These hierarchy levels are visually encoded through text indentation within the merged header cells; a convention invisible to the Table Transformer's bounding box predictions.

The Hierarchical Header Detector extracts the text content of each predicted header cell using EasyOCR (for scanned pages) or PyMuPDF text extraction (for digital pages). It then measures the left-indentation of the text in pixels relative to the cell's left boundary. K-means clustering ($k=5$, initialised with uniform spacing) is applied to the indentation values across all header cells in the table to assign discrete depth levels $D \in \{1, 2, 3, 4, 5\}$. This depth level is stored as the `hierarchy_level` attribute of each cell in the output JSON.

Multi-Page Continuity Stitcher (Lane C)

Government budget tables frequently span 10-50 pages, with each page containing a continuation of the same table. The Table Transformer treats each page independently and has no mechanism for detecting cross-page relationships. The Continuity Stitcher addresses this by comparing column-width signatures between consecutive pages.

For each page p , the column-width signature vector V_p is constructed by computing the relative width of each detected column as a fraction of the total table width. The cosine similarity between V_p and V_{p+1} is computed; if similarity exceeds 0.92, the two tables are declared continuations of each other. The first row of the continuation table is then inspected: if it is identified as a repeated header row (matching the header of the previous page with character-level Levenshtein distance < 0.15), it is stripped before merging the row index sequences. The stitched table is assigned a single `table_id` spanning both pages.

IPSAS Arithmetic Validator

As a final quality assurance step, the validator applies arithmetic consistency checks informed by the IPSAS budget document structure. For each table, it identifies numeric cells by applying a currency-normalisation regex (handling formats including "KSh 50,000,000", "Kshs. 50M", "50,000,000/-") and checks whether subtotal rows sum correctly to their parent totals within a tolerance of ± 1 (to account for rounding). Rows that fail arithmetic checks are flagged with a `validation_warning` attribute in the JSON output rather than discarded, preserving all data while alerting the downstream NLP module to treat these cells with lower confidence.

The performance of the arithmetic validator itself will be evaluated separately, with a target precision and recall of $>95\%$ for detecting inconsistencies in subtotal-to-total relationships. This ensures that the validation layer adds reliable quality control without introducing false positives that could erode trust in the extracted data.

3.1.7 Model Fine-Tuning for Domain Adaptation

The pre-trained Table Transformer was trained on PubTables-1M, a dataset of scientific paper tables with characteristics substantially different from Kenyan government documents. Scientific tables are typically narrow (3-8 columns), have simple 1-2 level headers, and rarely exceed one page. Government budget tables are wide (8-15 columns), have deep 5-level hierarchies, and span dozens of pages. Fine-tuning is therefore essential.

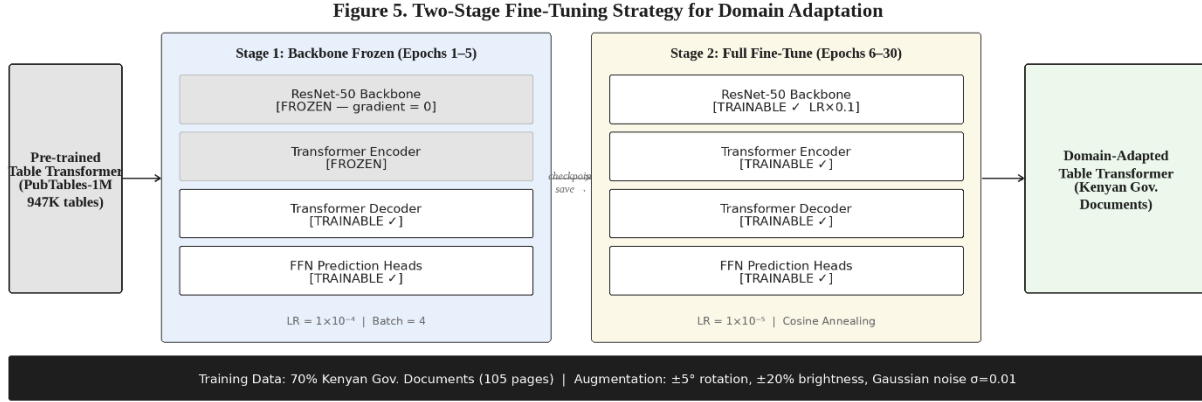


Figure 5: Two-Stage Fine-Tuning Strategy ; - Backbone Frozen (Stage 1) then Full Unfrozen (Stage 2)

Stage 1 - Backbone Frozen (Epochs 1-5)

In the first fine-tuning stage, the ResNet-50 backbone and transformer encoder weights are frozen (gradient = 0). Only the transformer decoder and FFN prediction heads are updated. This approach forces the model to adapt its query-based detection to the new domain while preserving the low-level visual features learned during pre-training. The learning rate is set to 1×10^{-4} with a batch size of 4 table images per step. Training for 5 epochs on the 70% training split (105 pages, approximately 800 table crops after augmentation) takes approximately 2 hours on a single NVIDIA RTX 3070 GPU.

Stage 2 - Full Unfreeze (Epochs 6-30)

In the second stage, all weights are unfrozen. The backbone is updated at one-tenth of the decoder learning rate (1×10^{-6}) to prevent catastrophic forgetting of the pre-trained low-level features, while the decoder and heads are updated at the full rate (1×10^{-5}). A cosine annealing schedule decays the learning rate to zero over 25 epochs. This stage allows the backbone to develop domain-specific visual representations (e.g., the appearance of IPSAS-formatted numerical cells) while retaining the generalisation capabilities acquired from PubTables-1M.

Data Augmentation

To mitigate the small size of the annotated training set, the following augmentations are applied online during training using the Albumentations library:

- Random rotation: ± 5 degrees (simulating scanner misalignment)
- Brightness adjustment: $\pm 20\%$ of the original pixel values
- Contrast adjustment: $\pm 15\%$
- Gaussian noise: $\sigma = 0.01$ (simulating low-quality scan texture)
- Random horizontal crop: up to 5% of page width (simulating margin variations)

Bounding box annotations are transformed consistently with each augmentation using Albumentations' built-in `bbox_params` support, ensuring that ground-truth labels remain spatially aligned after each transformation.

3.1.8 Stage 5: JSON Output Schema

The final output of the CV pipeline is a hierarchically structured JSON document that completely describes all tables extracted from the input PDF. Figure 6 illustrates the schema structure. The schema is designed to be the single source of truth for the downstream NLP module - every piece of information needed for entity extraction is contained within the JSON, with no requirement to re-access the original PDF.

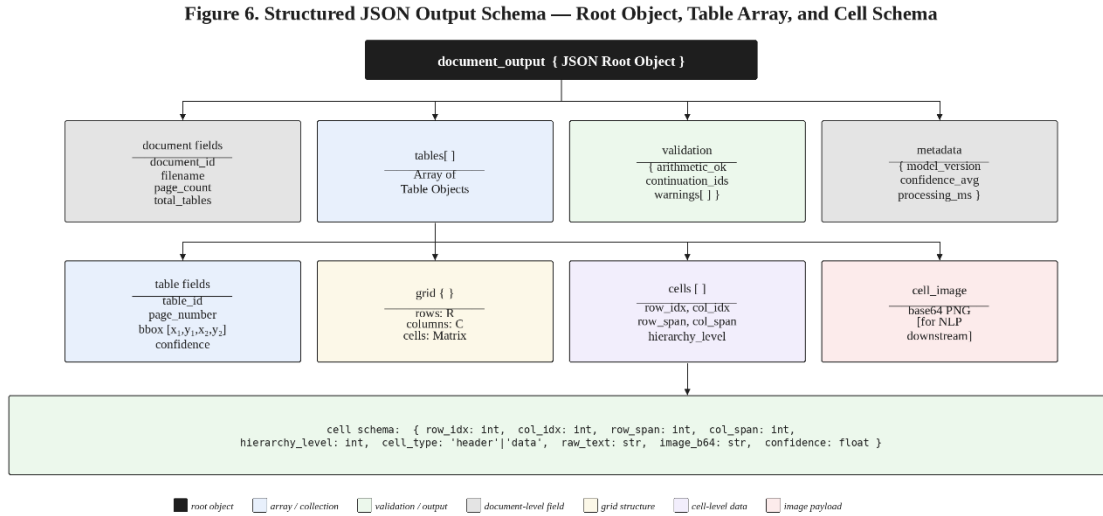


Figure 6: Structured JSON Output Schema ; - Root Object, Table Array, Cell Schema

The downstream LayoutLMv3-based NLP module will consume both the `raw_text` and the `image_b64` fields for each cell. The text provides a strong initial signal for entity recognition (e.g., identifying budget line items like "Road Maintenance"), while the image data allows the model to leverage subtle visual cues such as font emphasis, indentation, or background shading. These visual features are often critical for correctly classifying items in a hierarchy, where a "Sub-Programme" header may contain identical text to a "Programme" header but is distinguished by its visual presentation. The `raw_text` field is for reference only; NLP module will perform its own extraction from `image_b64`.

JSON Field	Description
document_id	Unique hash of source PDF filename + processing timestamp
page_count	Total number of pages in the document
tables[]	Array of all Table Objects extracted from the document
table_id	Unique table identifier (page_num + sequential index)

bbox [x1,y1,x2,y2]	Pixel-space bounding box of the table on the source page
grid{ rows, cols }	Dimensions of the reconstructed cell grid (integer R and C)
cells[]	Array of Cell Objects ordered by row-major traversal
row_idx / col_idx	Zero-based grid position of the cell's top-left corner
row_span / col_span	Number of rows/columns the cell spans (1 if not merged)
hierarchy_level	Hierarchical nesting level of header cells (1=top, 5=deepest)
cell_type	'header' for header rows/columns, 'data' for all other cells
raw_text	Normalised unicode string extracted from the cell region
image_b64	Base64-encoded PNG of the cell image region at 300 DPI
confidence	Model confidence score for cell boundary prediction (0-1)
validation{ }	Arithmetic validation results: is_valid, warnings[]

3.2 Technology Stack Summary

Library / Tool	Role in Pipeline
PyTorch 2.0+	Deep learning framework for model training and inference
HuggingFace Transformers	Pre-trained Table Transformer model loading and inference

pdf2image + Poppler	PDF to PIL Image conversion at configurable DPI
OpenCV 4.x	Skew detection (Hough transform) and image preprocessing
EasyOCR	OCR for cell text extraction from scanned pages
PyMuPDF (fitz)	Text extraction from digital PDFs; PDF metadata parsing
Albumentations	Online data augmentation with bbox-aware transforms
FastAPI	RESTful API exposing pipeline as POST /extract endpoint
Docker	Containerisation for reproducible deployment
pytest	Unit and integration testing for all pipeline components
NumPy / SciPy	Array operations, cosine similarity, k-means clustering

3.3 Data

Primary Data:

- 100 Kenyan government document pages (manual annotation)
- 150-200 table bounding boxes
- 15,000-20,000 cell-level structure labels

Annotation Process:

- Tool: LabelStudio
- Annotators: Self
- Quality control: Double-check 20% of samples
- Format: PubTables-1M compatible format

Data Split:

- Training: 70% (105 pages)
- Validation: 15% (23 pages)
- Testing: 15% (22 pages)

Data Augmentation:

- Random rotation: $\pm 5^\circ$
- Brightness/contrast adjustment
- Gaussian noise simulation

3.4 Evaluation Strategy and Metrics

Table Detection Metrics

Table detection performance is evaluated using standard object detection metrics computed on the held-out 15% test split (approximately 22 pages). Precision, Recall, and F1 score are reported at IoU thresholds of 0.50 and 0.75. The stricter 0.75 threshold is more relevant for the WaziGov use case, as an inaccurate bounding box causes cell grid misalignment in the downstream structure recognition stage. The target is $F1 > 0.90$ at $IoU=0.50$ and $F1 > 0.80$ at $IoU=0.75$.

Structure Recognition Metrics

Structure recognition is evaluated using the Grid Table Similarity (GriTS) family of metrics introduced by Smock et al. (2022). GriTS-Top measures whether cells are correctly localised (bounding box accuracy). GriTS-Con measures whether the adjacency relationships between cells are correct (structural accuracy). GriTS-Loc measures both simultaneously. The primary target metric is $GriTS-Con > 0.85$, as correct cell adjacency is the minimum requirement for the NLP module to correctly associate text entities with their parent columns.

Acknowledging the Fidelity Debate: Recent work by Liu (2025) identifies a potential "Fidelity Gap" where structural similarity metrics may not fully capture information preservation. While GriTS remains the standard metric for table structure evaluation (Smock et al., 2022; Taverne et al., 2025), for this project GriTS-Con is appropriate as our primary metric because the downstream NLP module requires correct cell adjacency; precisely what GriTS-Con measures. However, we will also conduct qualitative error analysis and leverage the arithmetic validation layer (Section 3.6.4) to ensure structural accuracy translates to information fidelity for budget transparency use cases.

Merged Cell and Hierarchy Metrics

Merged cell detection is evaluated using Span Accuracy: the fraction of ground-truth merged cells for which both the row-span and col-span values are correctly predicted. Hierarchical header detection is evaluated using Depth Level Accuracy: the fraction of header cells assigned the correct depth level compared to the manually annotated ground truth. Targets are 80% Span Accuracy and 75% Depth Level Accuracy.

Baseline Comparisons

The fine-tuned Table Transformer is compared against three baselines: (1) pdfplumber (rule-based PDF text extraction), (2) Camelot (lattice and stream modes), and (3) the zero-shot (unfine-tuned) Table Transformer. This three-way comparison simultaneously demonstrates the superiority of deep learning over rule-based methods and quantifies the marginal value of domain-specific fine-tuning.

Anticipated Failure Modes and Mitigation Strategies

While the proposed system is designed for robustness, several specific edge cases may challenge its performance. Proactive identification of these failure modes informs both the evaluation strategy and potential mitigation approaches.

1. **Faint or 'Zebra' Shading in Data Rows:** Many government documents use alternating light grey shading to improve row readability. The Table Transformer's structure model, trained primarily on unshaded scientific tables, might misinterpret this consistent low-contrast pattern as actual table rows or columns, leading to an over-segmented grid. **Mitigation:** The fine-tuning dataset will include representative examples of shaded tables. Additionally, a post-processing heuristic will be developed to detect and ignore repeating, low-amplitude intensity variations that do not align with text baselines.
2. **Overlapping Table Annotations and Watermarks:** Scanned documents often contain faint background watermarks ("SPECIMEN" or departmental crests) that intersect with table regions. These watermarks create visual noise that could confuse the detection model or be misclassified as table structure elements. **Mitigation:** The contrast normalization (CLAHE) step in preprocessing is designed to suppress low-contrast background artefacts. The model will also be fine-tuned on pages with simulated watermark noise to learn invariance to such patterns.
3. **Tables with Irregular or Missing Cell Boundaries:** Some budget documents use leader dots (.....) or whitespace to imply cell separation instead of visible lines. The model must infer structure from text alignment alone. **Mitigation:** The Hierarchical Header Detector's indentation analysis and the grid construction algorithm's reliance on centre coordinates of predicted rows/columns are designed to handle boundary-less cells. The fine-tuning data will include a stratified sample of such "borderless" tables.
4. **Multi-Page Continuity Failures:** Tables that continue across pages but with slight column width variations (due to scanning artefacts or formatting changes) may fall below the 0.92 cosine similarity threshold. **Mitigation:** The continuity stitcher will implement a fallback mechanism using header text matching (Levenshtein distance) when column-width similarity falls in the borderline range (0.85-0.92), following the multi-page merging principles validated by Liu et al. (2025).

A dedicated error analysis phase (Objective x) will quantify the frequency of these and other failure modes, guiding further refinements.

3.5 Expected Outcomes

Deliverables:

1. Fine-tuned Table Transformer models (detection + structure)
2. Annotated dataset of Kenyan government tables (public release)
3. Production-ready Python package with REST API

4. Technical documentation and API specifications

Performance Targets:

- Table detection: >90% F1 @ IoU=0.5
- Structure recognition: >85% GriTS-Con
- Processing speed: <2 seconds per page on GPU

Contributions:

- First benchmark dataset for African government documents
- Novel multi-page continuity algorithm
- IPSAS-specific arithmetic validation layer
- Open-source release enabling reproducibility

3.6 Ethical Considerations

Data Privacy and Security:

1. **Public Data Only:**
 - All documents are publicly available government reports
 - No personal data or confidential information
 - Compliance with Kenya Data Protection Act (2019)
2. **Data Handling:**
 - Secure storage of documents during processing
 - No retention of personal identifiable information
 - Anonymization of any incidental personal data

Algorithmic Bias:

1. **Model Bias:**
 - Pre-trained models may favour certain table layouts
 - Mitigation: Fine-tune on diverse Kenyan documents
 - Regular bias audits on different counties
2. **Representation:**
 - Ensure dataset includes diverse document types
 - Test on all 47 counties (urban and rural)

Transparency:

1. **Open Source:**
 - Code released under MIT license
 - Models and weights publicly available

- Documentation of limitations and failure modes
- 2. **Reproducibility:**
 - Detailed methodology documentation
 - Version control of all components
 - Fixed random seeds for experiments

Environmental Considerations:

- Optimize model inference to reduce GPU energy consumption
- Use efficient batch processing
- Consider CPU-only deployment for production

Trust and Accountability:

- Clear documentation of accuracy limitations
- Error reporting mechanisms
- Regular updates based on user feedback

4. CONCLUSION

This project addresses a critical technical challenge in document understanding: automated extraction of complex table structures from government financial documents. By leveraging state-of-the-art transformer-based architectures (DETR, Table Transformer), the Computer Vision module will enable accurate, pixel-perfect table detection and structure recognition that defeats traditional PDF parsing methods.

The system's ability to handle merged cells, hierarchical headers, and multi-page continuity makes it uniquely suited for IPSAS-compliant government budgets. With target accuracies exceeding 90% F1 for detection and 85% GriTS for structure recognition, the module will provide a robust foundation for downstream NLP processing in the WaziGov transparency pipeline.

Key technical contributions include domain adaptation strategies for government documents, a novel multi-page continuity detection algorithm, and the first publicly available benchmark dataset of annotated Kenyan government tables. The modular architecture with RESTful API ensures seamless integration with NLP and geospatial components.

Beyond technical achievements, this work advances document AI research in African contexts, addressing the gap in evaluation datasets and methods for non-Western document formats. The open-source release of code, models, and datasets will enable reproducibility and further research in government transparency technologies.

5. REFERENCES

- Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-end object detection with transformers. In *Proceedings of the European Conference on Computer Vision (ECCV)* (pp. 213-229). Springer.
- Chi, Z., Huang, H., Xu, H. D., Yu, H., Yin, W., & Mao, X. L. (2019). Complicated table structure recognition. *arXiv*. <https://arxiv.org/abs/1908.04729>
- Giovannini, S., & Marinai, S. (2025). A survey on reading order, table of contents, and structure extraction in document analysis. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops* (pp. 7585–7594). IEEE.
- Huang, Y., Lv, T., Cui, L., Lu, Y., & Wei, F. (2022). LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proceedings of the 30th ACM International Conference on Multimedia* (pp. 4083-4091). ACM.
- Li, M., Cui, L., Huang, S., Wei, F., Zhou, M., & Li, Z. (2021). TableFormer: Table structure understanding with transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2021* (pp. 4614-4623). IEEE/CVF.
- Liu, Y. (2025). From structural similarity to information fidelity: A new paradigm for evaluating table recognition. Hong Kong University of Science and Technology.
- Liu, Y., Chen, J., Zhang, R., & Wang, H. (2025). MonkeyOCR v1.5 technical report: Unlocking robust document parsing for complex patterns. *arXiv*. <https://arxiv.org/abs/2511.10390>
- Maurya, S., & Prajapati, A. (2025). Table structure recognition: A review of current approaches and future opportunities. In *International Conference on Advancing Technology in Engineering and Science (ICATES 2025)*. [Publisher information, e.g., IEEE].
- Nunes, G., Oliveira, D., & Santos, R. (2025). Benchmarking table extraction: Multimodal LLMs vs traditional OCR. In *Proceedings of the 1st Joint Workshop on Large Language Models and Structure Modeling (XLLM 2025)* (pp. 8–15). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2025.xllm-1.2>
- Paliwal, S. S., Vishwanath, D., Rahul, R., Sharma, M., & Vig, L. (2020). TableNet: Deep learning model for end-to-end table detection and tabular data extraction from scanned document images. In *2019 International Conference on Document Analysis and Recognition (ICDAR), 2019* (pp. 128-133). IEEE.
- Qiao, L., Li, Z., Cheng, Z., Zhang, P., Pu, S., Niu, Y., Ren, W., Tan, W., & Wu, F. (2021). LGPMA: Complicated table structure recognition with local and global pyramid mask alignment. In *2021 International Conference on Document Analysis and Recognition (ICDAR)* (pp. 99-114). Springer.
- Saha, R., Mondal, A., & Jawahar, C. V. (2019). Graphical object detection in document images. In *2019 International Conference on Document Analysis and Recognition (ICDAR), 2019* (pp. 51-58). IEEE.

Schreiber, S., Agne, S., Wolf, I., Dengel, A., & Ahmed, S. (2017). DeepDeSRT: Deep learning for detection and structure recognition of tables in document images. In *2017 14th IAPR International Conference on Document Analysis and Recognition (ICDAR), 2017* (Vol. 1, pp. 1162-1167). IEEE.

Smock, B., Pesala, R., & Abraham, R. (2022). PubTables-1M: Towards comprehensive table extraction from unstructured documents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022** (pp. 4634-4642). IEEE/CVF.

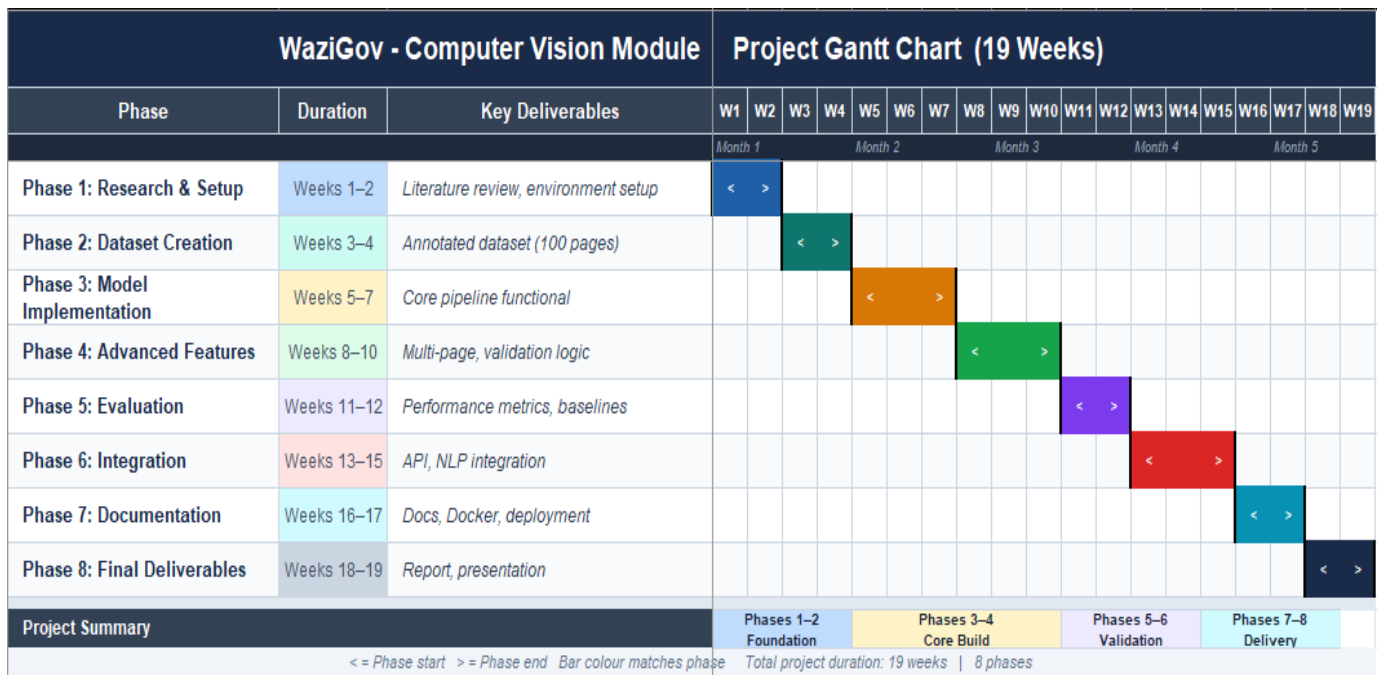
Thomas, E., Coustaty, M., Joseph, A., Deloin, G., Carel, E., Poulain d'Andecy, V., & Ogier, J.-M. (2025). RAPTOR: Refined approach for product table object recognition. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision Workshops (WACVW)* (pp. 1333-1342). IEEE. <https://doi.org/10.1109/WACVW65960.2025.00147>

Zhang, Y., Wang, L., & Chen, X. (2025). HTAN: A hierarchical table-aware network for complex table visual understanding. In *Proceedings of the 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025*. IEEE/CVF.

6. APPENDICES

Appendix 1: Project Plan and Timeline

The following Gantt chart illustrates the 34-week project timeline with task dependencies, milestones, and deliverables:



Appendix 2: Budget

Detailed Budget Breakdown:

Category	Item	Quantity	Unit Cost (KSh)	Total (KSh)
Computing	GPU Cloud Credits	4 months	1,250/month	5,000
Computing	Storage (Cloud)	50GB	50/GB	2,500
Data	Manual Annotation	100 pages	30/page	3,000
Testing	Test Computing	20 hours	125/hour	2,500
Deployment	Container Hosting	3 months	1,667/month	5,000
Misc	Documentation	Lump sum	-	2,000
Contingency	Buffer (10%)	Lump sum	-	2,500
	TOTAL			22,500